The captured packets can then be stored to a variable and the summary can be printed using the 'nsummary()' function as demonstrated below:

```
>>> s=_
>>> s.nsummary()
0000 Ether / IP / TCP 192.168.111.239:49152 >
192.168.111.81:59850 FPA / Raw
0001 Ether / IP / TCP 192.168.111.81:59850 >
192.168.111.239:49152 FA
0002 Ether / IP / TCP 192.168.111.81:59858 >
192.168.111.239:49152 S
0003 Ether / IP / TCP 192.168.111.239:49152 >
192.168.111.81:59850 A
...
```

You can also view a specific packet using the array subscript syntax:

```
>>> s[1]
<Ether  dst=0c:91:60:5b:27:ec src=58:6c:25:ed:5d:d7 type=IPv4
|<IP  version=4 ihl=5 tos=0x0 len=52 id=37178 flags=DF
frag=0 ttl=64 proto=tcp chksum=0x48f8 src=192.168.111.81
dst=192.168.111.239 |<TCP  sport=59850 dport=49152
seq=2775654087 ack=1975887132 dataofs=8 reserved=0 flags=FA
window=501 chksum=0x60b8 urgptr=0 options=[('NOP', None),
('NOP', None), ('Timestamp', (937481726, 627853))] |>>>
```

The 'hexdump()' function can output the hexadecimal values of the packets along with their ASCII text for reference.

```
>>> s.hexdump()
0000 01:10:59.618890 Ether / IP / TCP 192.168.111.239:49152 >
192.168.111.81:59850 FPA / Raw
0000  58 6C 25 ED 5D D7 0C 91 60 5B 27 EC 08 00 45 00
Xl%.]...`['...E.
0010  03 09 0F 2A 40 00 40 06 C8 33 C0 A8 6F EF C0 A8
...*@.@..3..o...
0020  6F 51 C0 00 E9 CA 75 C5 A2 46 A5 71 1E C7 80 19
oQ....u..F.q....
0030  01 C5 94 64 00 00 01 01 08 0A 00 09 94 8D 37 E0
...d..........7.
0040  D5 C6 72 76 69 63 65 49 64 3A 52 65 6E 64 65 72
..rviceId:Render
0050  69 6E 67 43 6F 6E 74 72 6F 6C 3C 2F 73 65 72 76
ingControl</serv
0060  69 63 65 49 64 3E 0D 0A 3C 53 43 50 44 55 52 4C
iceId>..<SCPDURL
0070  3E 2F 72 63 72 2E 78 6D 6C 3C 2F 53 43 50 44 55  >/rcr.
xml</SCPDU
0080  52 4C 3E 0D 0A 3C 63 6F 6E 74 72 6F 6C 55 52 4C
RL>..<controlURL
...
```

If you have multiple network interfaces, you can explicitly specify the 'iface' attribute to capture the packets as demonstrated below:

```
>>> sniff(iface="wlp0s20f3", prn=lambda x: x.show())
###[ Ethernet ]###
  dst= 33:33:00:00:00:fb
  src= 72:71:d9:df:a3:b1
  type= IPv6
###[ IPv6 ]###
     version= 6
     tc= 0
     fl= 197632
     plen= 262
     nh= UDP
     hlim= 255
     src= fe80::840:4b79:ad35:c4dd
     dst= ff02::fb
###[ UDP ]###
     sport= mdns
     dport= mdns
     len= 262
     chksum= 0xa5c7
...
```

PCAP

The capture packets can be written to a file using the 'wrpcap()' function.

```
>>> wrpcap("output.cap", s)
```

The packets can also be read from a file using the 'rdpcap' function as follows:

```
>>> data = rdpcap("output.cap")
```

Using the array subscription, you can get the detailed output for a specific packet:

```
>>> data[2]
<Ether  dst=0c:91:60:5b:27:ec src=58:6c:25:ed:5d:d7 type=IPv4
|<IP  version=4 ihl=5 tos=0x0 len=60 id=44535 flags=DF
frag=0 ttl=64 proto=tcp chksum=0x2c33 src=192.168.111.81
dst=192.168.111.239 |<TCP  sport=59858 dport=49152
seq=3260427060 ack=0 dataofs=10 reserved=0 flags=S
window=64240 chksum=0x60c0 urgptr=0 options=[('MSS', 1460),
('SAckOK', b''), ('Timestamp', (937481726, 0)), ('NOP',
None), ('WScale', 7)] |>>>
```